Sesión 19: Patrón BLoC / Cubit: Principios (Separación de Lógica de Negocio y Presentación mediante Flujos de Eventos y Estados)

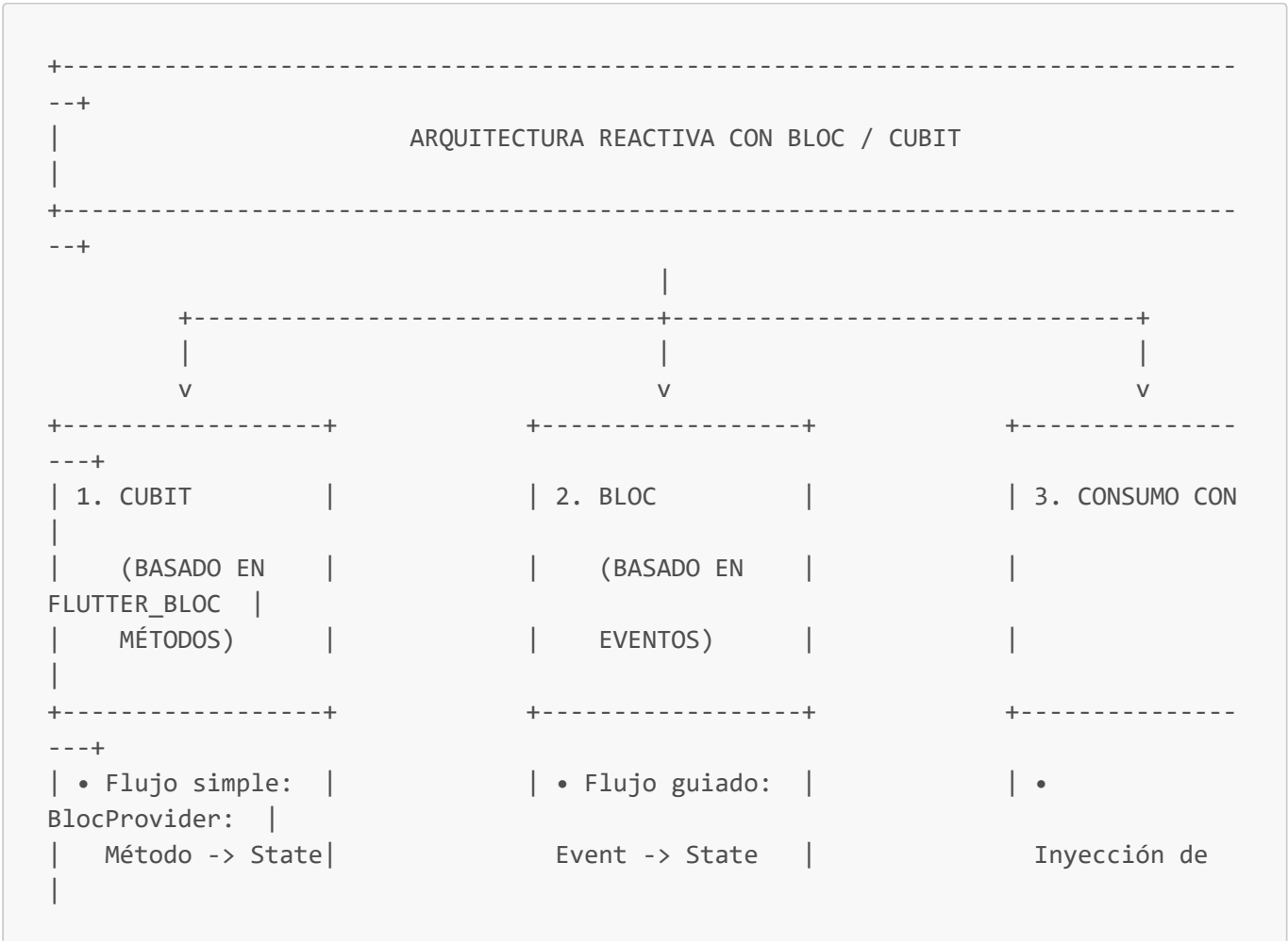
Introducción

El patrón BLoC (*Business Logic Component*), creado por Google, es una de las arquitecturas más extendidas en el ecosistema de Flutter para garantizar la estricta separación de responsabilidades entre la interfaz de usuario y la lógica de negocio. Basado en el paradigma de programación reactiva y la gestión de flujos (`Streams`), BLoC convierte los eventos enviados por la interfaz en nuevos estados inmutables que la capa de presentación re-renderiza.

En esta sesión estudiaremos los fundamentos del patrón BLoC y su variante simplificada, `Cubit`. Analizaremos las diferencias operativas entre invocar métodos directamente y procesar eventos asíncronos mediante transmisores (*Event Transformers*), sentando las bases para construir aplicaciones escalables, mantenibles y fácilmente testeables.

Conceptos y Definiciones

El principio rector de BLoC/Cubit es aislar por completo el código de la UI del procesamiento de datos, convirtiendo la interfaz en una representación puramente pasiva de los estados recibidos.



```
| • Menos verboso, |            | • Registro vía   |              dependencias.
|
|   ideal para UI  |              on<Evento>().    |            | • BlocBuilder:
|
|   directa.       |            | • Soporta        |              Redibujado UI.
|
| • emite() nuevos |              transformación   |            | •
BlocListener:  |
|   estados.       |              de eventos.      |              Efectos puros.
|
+------------------+           +------------------+           +---------------
---+
```

1. Cubit: Lógica de Negocio Simplificada basada en Métodos

- `Cubit<State>` es una subclase simplificada de BLoC que no utiliza eventos.
- Expone funciones o métodos que pueden ser llamados directamente desde la interfaz de usuario.
- Cuando un método ejecuta su lógica, invoca la función interna `emit(nuevoEstado)` para notificar a los suscriptores.
- **Uso ideal:** Pantallas con flujos directos, formularios sencillos, conmutadores de tema o lógica de bajo acoplamiento.

---

2. BLoC: Control Guiado por Eventos (*Event-Driven Architecture*)

- `Bloc<Event, State>` utiliza eventos para desencadenar transformaciones de estado en lugar de llamadas a métodos directos.
- Recibe `Events` como entradas (*inputs*) a través de un `Stream` y emite `States` como salidas (*outputs*).
- Los manejadores de eventos se registran explícitamente mediante la sintaxis `on<Evento>((event, emit) async => ...)` dentro del constructor.
- **Ventajas avanzadas:** Permite el uso de transformadores de eventos (*Event Transformers*) como `debounce()`, `throttle()` o `droppable()`, ideales para barras de búsqueda asíncronas, protección contra doble clic o peticiones en cola.

---

3. Componentes Fundamentales del Paquete `flutter_bloc`

- **`BlocProvider`:** Widget encargado de instanciar e inyectar un BLoC o Cubit en el árbol de widgets mediante `context.read<MiBloc>()`.
- **`BlocBuilder`:** Widget reactivo que escucha las emisiones de estado y re-ejecuta su función creadora (`builder`) únicamente cuando se emite un nuevo estado.
- **`BlocListener`:** Escucha estados para ejecutar efectos secundarios que no implican rediseño gráfico (ej. mostrar un `SnackBar`, un diálogo de confirmación o realizar una navegación).
- **`BlocConsumer`:** Combina las capacidades de `BlocBuilder` y `BlocListener` en un único componente.

Ejemplo Completo con Código Fuente: Sistema de Autenticación con BLoC y Cubit

El siguiente ejemplo implementa una arquitectura híbrida que combina un `Cubit` para el control de la contraseña en tiempo real y un `Bloc` basado en eventos para procesar el flujo asíncrono de autenticación del usuario.

> **Nota de dependencia:** Requiere incluir `flutter_bloc: ^8.1.3` y `equatable: ^2.0.5` en el archivo `pubspec.yaml`.

Código Fuente Explicado (`lib/main.dart`)

```dart
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:equatable/equatable.dart';

void main() {
  runApp(const DemoBlocApp());
}

// ============================================================================
// 1. CUBIT: Control de Visibilidad de Contraseña (Lógica Directa)
// ============================================================================

class VisibilidadPasswordCubit extends Cubit<bool> {
  // Estado inicial: contraseña oculta (true = obscureText)
  VisibilidadPasswordCubit() : super(true);

  void alternarVisibilidad() => emit(!state);
}

// ============================================================================
// 2. BLOC: Sistema de Autenticación Basado en Eventos
// ============================================================================

// --- EVENTOS ---
abstract class AuthEvent extends Equatable {
  const AuthEvent();
  @override
  List<Object?> get props => [];
}

class LoginSolicitadoEvent extends AuthEvent {
  final String usuario;
  final String password;

  const LoginSolicitadoEvent(this.usuario, this.password);

  @override
  List<Object?> get props => [usuario, password];
}

class LogoutSolicitadoEvent extends AuthEvent {}

// --- ESTADOS ---
```

```dart
abstract class AuthState extends Equatable {
  const AuthState();
  @override
  List<Object?> get props => [];
}

class AuthInicialState extends AuthState {}
class AuthCargandoState extends AuthState {}
class AuthExitosoState extends AuthState {
  final String nombreUsuario;
  const AuthExitosoState(this.nombreUsuario);

  @override
  List<Object?> get props => [nombreUsuario];
}
class AuthErrorState extends AuthState {
  final String mensaje;
  const AuthErrorState(this.mensaje);

  @override
  List<Object?> get props => [mensaje];
}

// --- BLOC IMPLEMENTACIÓN ---
class AuthBloc extends Bloc<AuthEvent, AuthState> {
  AuthBloc() : super(AuthInicialState()) {
    // Registro explícito del manejador de evento mediante on<E>()
    on<LoginSolicitadoEvent>(_onLoginSolicitado);
    on<LogoutSolicitadoEvent>(_onLogoutSolicitado);
  }

  Future<void> _onLoginSolicitado(
    LoginSolicitadoEvent event,
    Emitter<AuthState> emit,
  ) async {
    emit(AuthCargandoState());

    // Simulación de respuesta asíncrona de servidor
    await Future.delayed(const Duration(seconds: 2));

    if (event.usuario == 'ozzy' && event.password == '123456') {
      emit(const AuthExitosoState('Profesor Ozzy'));
    } else {
      emit(const AuthErrorState('Credenciales incorrectas. Intenta de nuevo.'));
    }
  }

  void _onLogoutSolicitado(
    LogoutSolicitadoEvent event,
    Emitter<AuthState> emit,
  ) {
    emit(AuthInicialState());
  }
}
```

```dart
// ============================================================================
// INTERFAZ DE USUARIO CON FLUTTER_BLOC
// ============================================================================

class DemoBlocApp extends StatelessWidget {
  const DemoBlocApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MultiBlocProvider(
      providers: [
        BlocProvider<AuthBloc>(create: (context) => AuthBloc()),
        BlocProvider<VisibilidadPasswordCubit>(
          create: (context) => VisibilidadPasswordCubit(),
        ),
      ],
      child: MaterialApp(
        title: 'ESCOM - Arquitectura BLoC',
        debugShowCheckedModeBanner: false,
        theme: ThemeData(
          useMaterial3: true,
          colorScheme: ColorScheme.fromSeed(
            seedColor: const Color(0xFF6B112D), // Guinda Institucional
          ),
        ),
        home: const PantallaLogin(),
      ),
    );
  }
}

class PantallaLogin extends StatefulWidget {
  const PantallaLogin({super.key});

  @override
  State<PantallaLogin> createState() => _PantallaLoginState();
}

class _PantallaLoginState extends State<PantallaLogin> {
  final _usuarioController = TextEditingController();
  final _passwordController = TextEditingController();

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Inicio de Sesión BLoC / Cubit'),
        backgroundColor: Theme.of(context).colorScheme.primary,
        foregroundColor: Colors.white,
      ),
      body: Padding(
        padding: const EdgeInsets.all(20.0),
        child: BlocConsumer<AuthBloc, AuthState>(
```

```
            // BlocConsumer permite escuchar efectos y construir UI simultáneamente
            listener: (context, state) {
              if (state is AuthErrorState) {
                ScaffoldMessenger.of(context).showSnackBar(
                  SnackBar(
                    content: Text(state.mensaje),
                    backgroundColor: Colors.red,
                  ),
                );
              }
            },
            builder: (context, state) {
              if (state is AuthCargandoState) {
                return const Center(
                  child: Column(
                    mainAxisAlignment: MainAxisAlignment.center,
                    children: [
                      CircularProgressIndicator(),
                      SizedBox(height: 12),
                      Text('Validando credenciales en el servidor...'),
                    ],
                  ),
                );
              }

              if (state is AuthExitosoState) {
                return Center(
                  child: Column(
                    mainAxisAlignment: MainAxisAlignment.center,
                    children: [
                      const Icon(Icons.check_circle, color: Colors.green, size: 64),
                      const SizedBox(height: 12),
                      Text(
                        '¡Bienvenido, ${state.nombreUsuario}!',
                        style: const TextStyle(fontSize: 20, fontWeight:
FontWeight.bold),
                      ),
                      const SizedBox(height: 20),
                      ElevatedButton(
                        onPressed: () {
                          // Disparo de evento al BLoC
                          context.read<AuthBloc>().add(LogoutSolicitadoEvent());
                        },
                        child: const Text('Cerrar Sesión'),
                      ),
                    ],
                  ),
                );
              }

              // Estado por defecto: Formulario (AuthInicialState o AuthErrorState)
              return Column(
                mainAxisAlignment: MainAxisAlignment.center,
                children: [
```

```
                TextField(
                  controller: _usuarioController,
                  decoration: const InputDecoration(
                    labelText: 'Usuario',
                    prefixIcon: Icon(Icons.person),
                    border: OutlineInputBorder(),
                  ),
                ),
                const SizedBox(height: 16),

                // CONSUMO DE CUBIT PARA OCULTAR/MOSTRAR CONTRASEÑA
                BlocBuilder<VisibilidadPasswordCubit, bool>(
                  builder: (context, ocultarPassword) {
                    return TextField(
                      controller: _passwordController,
                      obscureText: ocultarPassword,
                      decoration: InputDecoration(
                        labelText: 'Contraseña',
                        prefixIcon: const Icon(Icons.lock),
                        border: const OutlineInputBorder(),
                        suffixIcon: IconButton(
                          icon: Icon(
                            ocultarPassword
                                ? Icons.visibility
                                : Icons.visibility_off,
                          ),
                          onPressed: () {
                            // Invocación directa del método en Cubit
                            context
                                .read<VisibilidadPasswordCubit>()
                                .alternarVisibilidad();
                          },
                        ),
                      ),
                    );
                  },
                ),
                const SizedBox(height: 24),

                SizedBox(
                  width: double.infinity,
                  height: 48,
                  child: ElevatedButton(
                    onPressed: () {
                      // Disparo de evento con datos al AuthBloc
                      context.read<AuthBloc>().add(
                            LoginSolicitadoEvent(
                              _usuarioController.text,
                              _passwordController.text,
                            ),
                          );
                    },
                    child: const Text('Ingresar'),
                  ),
```

```
                ),
              ],
            );
          },
        ),
      ),
    );
  }
}
```

Prácticas Guiadas / Laboratorio

Práctica 1: Implementación de un Contador con `Cubit` e Inmutabilidad

1. Crea una clase `ContadorCubit` que herede de `Cubit<int>` iniciando en 0.
2. Añade métodos `incrementar()` y `decrementar()` utilizando la función `emit()`.
3. Conecta la UI utilizando un `BlocBuilder` e implementa dos botones de acción flotantes para modificar el valor del estado.

Práctica 2: Transformación de Eventos en `BLoC` mediante `EventTransformer`

1. Investiga el paquete `bloc_concurrency`.
2. Modifica el `AuthBloc` para registrar el evento de inicio de sesión utilizando el transformador `droppable()`: `on<LoginSolicitadoEvent>(_onLoginSolicitado, transformer: droppable())`.
3. Comprueba mediante pruebas rápidas de interacción que los clics repetidos sobre el botón "Ingresar" se descartan mientras la primera petición siga en estado de carga.

Resumen y Puntos Clave

- **Desacoplamiento Absoluto:** El patrón BLoC separa totalmente la UI de la lógica de negocio mediante el intercambio exclusivo de entradas (eventos) y salidas (estados).
- **Cubit vs BLoC:** `Cubit` es ideal para lógica directa sin eventos (invocación vía métodos); `BLoC` se recomienda para flujos asíncronos avanzados accionados por eventos estructurados.
- **`flutter_bloc` Scaffolding:** `BlocProvider` inyecta instancias; `BlocBuilder` redibuja vistas; `BlocListener` ejecuta efectos secundarios; `BlocConsumer` integra ambos patrones.
- **Uso de `Equatable`:** Facilita la comparación por valor de las instancias de estado, evitando reconstrucciones innecesarias en la UI cuando el estado no cambia.

Bibliografía y Referencias

1. **Angelov, F.** (2024). *Bloc State Management Library Documentation*. Recuperado de: https://bloclibrary.dev
2. **Google LLC.** (2024). *State Management - BLoC Pattern in Flutter*. Recuperado de: https://docs.flutter.dev/data-and-backend/state-mgmt/options#bloc--rx

3. **Moedano, B.** (2023). *Desarrollo de Aplicaciones Móviles Adaptativas con Flutter*. Editorial Alfaomega. ISBN: 978-6075389102.